Q1 Master Theorem: Four Subproblems 4 marks

Analyze $T(n) = 4T(n/2) + n^2 \log n$ using the Master Theorem. Identify a, b, and f(n), state whether a standard case applies, and justify the asymptotic bound.

Your Answer

Step 1: Identify the values
$a = 4, b = 2, f(n) = n^2 \log n$
Step 2: Calculate
$n^{(\log_2 4)} = n^2$
Step 3: Identify the Master Theorem case
$f(n) = n^2 \log n = \Theta(n^2 \log^1 n)$
None of the three basic Master Theorem cases applies directly. The extended Master Theorem Case 2 applies.
Step 4: Final asymptotic bound
$T(n) = \Theta(n^2 (\log n)^2)$
Justification: The extended Case 2 adds one extra logarithmic factor, giving $T(n) = \Theta(n^2 (\log n)^2)$.

Save Answer Next →

Q2 Master Theorem: Unequal Growth 4 marks

Analyze $T(n) = 3T(n/2) + n$ using the Master Theorem, where applicable. Explain which term dominates.

Your Answer

Step 1: Identify the values
$a = 3, b = 2, f(n) = n$

Step 2: Calculate
$n^{(\log_b a)} = n^{(\log_2 3)} \approx n^{1.585}$

Step 3: Compare the growth rates
$f(n) = n$
n grows slower than $n^{(\log_2 3)}$.
Therefore, $n^{(\log_2 3)}$ dominates.

Step 4: Apply the Master Theorem
Since $f(n) = O(n^{(\log_2 3 - \varepsilon)})$ for some $\varepsilon > 0$, Master Theorem Case 1 applies.

Final Answer
$T(n) = \Theta(n^{(\log_2 3)})$s
Justification: The recursive term dominates the non-recursive term n, so Case 1 applies.

Save Answer Next →

Q3 Master Theorem: Square-Root Work 4 marks

Find the asymptotic complexity of $T(n) = 2T(n/4) + \sqrt{n}$. State the parameters and justify the applicable case.

Your Answer

Step 1: Identify the parameters

a = 2, b = 4, f(n) = √n = n^(1/2)

Step 2: Calculate

n^(log_b a) = n^(log₄ 2) = n^(1/2)

Step 3: Compare the terms

f(n) = n^(1/2) = Θ(n^(log₄ 2))

Both terms have the same growth rate. Therefore, Master Theorem Case 2 applies.

Step 4: Final Answer

T(n) = Θ(√n log n)

Justification: Since f(n) = Θ(n^(log_b a)), Case 2 gives T(n) = Θ(n^(log_b a) log n) = Θ(√n log n).

Save Answer Next →

Q4 Master Theorem: Five Subproblems 4 marks

A divide-and-conquer algorithm makes 5 recursive calls on subproblems of size n/3 and performs $\Theta(n^2)$ extra work. Determine its asymptotic time complexity.

Your Answer

Step 1: Identify the parameters
a = 5, b = 3, f(n) = $O(n^2)$

Step 2: Calculate
n^(log_b a) = n^(log₃ 5) ≈ n^1.465

Step 3: Compare the terms
f(n) = $O(n^2)$
Since n^2 grows faster than n^(log₃ 5), the extra work dominates.

Step 4: Apply the Master Theorem
Case 3 applies, provided the regularity condition holds.

Final Answer
T(n) = $\Theta(n^2)$
Justification: The non-recursive work $O(n^2)$ grows faster than the recursive contribution, so the total complexity is $\Theta(n^2)$.

Save Answer Next →

Q5 Master Theorem: Logarithmic Factor 4 marks

Analyze $T(n) = 16T(n/4) + n^2/\log n$. Discuss whether the standard Master Theorem applies directly and provide a justified bound.

Your Answer

Step 1: Identify the parameters
$a = 16$, $b = 4$, $f(n) = n^2 / \log n$

Step 2: Calculate
$n^{(\log_b a)} = n^{(\log_4 16)} = n^2$

Step 3: Compare the terms
$f(n) = n^2 / \log n$
The non-recursive term is smaller than n^2 by a logarithmic factor.

Step 4: Apply the Master Theorem
The standard Master Theorem does not apply directly, because $f(n) = n^2 / \log n$ does not satisfy any of the three basic cases.
Using an extended Master Theorem or a recursion-tree analysis, the total cost is dominated by the contributions across the recursion levels.

Final Answer
$T(n) = \Theta(n^2 \log \log n)$
Justification: Each recursion level contributes approximately $n^2 / \log(n / 4^i)$. Summing these contributions over the levels gives $\Theta(n^2 \log \log n)$.

Save Answer Next →

Q6 Substitution: Linearithmic Recurrence 4 marks

Use the substitution method to prove an asymptotic bound for $T(n) = 2T(n/2) + n$. State an induction hypothesis and base case.

Your Answer

Step 1: Given recurrence
$T(n) = 2T(n/2) + n$
Assume the asymptotic bound is $T(n) = O(n \log n)$.

Step 2: Induction hypothesis
Assume $T(n/2) \le c(n/2) \log_2(n/2)$, where $c > 0$.

Step 3: Substitute
$T(n) \le 2[c(n/2) \log_2(n/2)] + n$
$T(n) \le cn(\log_2 n - 1) + n$
$T(n) \le cn \log_2 n - cn + n$
$T(n) \le cn \log_2 n$, for $c \ge 1$.

Step 4: Base case
For $n = 1$, $T(1) = d$, where d is a constant. Choose c large enough to satisfy the base case.

Final Answer
$T(n) = O(n \log n)$
Using a matching lower bound, $T(n) = \Theta(n \log n)$.

Justification: The substitution method confirms the upper bound $O(n \log n)$, and the recurrence has a matching lower bound.

Save Answer Next →

Q7 Substitution: Decreasing by One 4 marks

Use substitution or expansion to establish a tight asymptotic bound for $T(n) = T(n-1) + n$, assuming $T(1)=1$.

Your Answer

Step 1: Given recurrence
$T(n) = T(n - 1) + n$, where $T(1) = 1$.

Step 2: Expand the recurrence
$T(n) = T(n - 2) + (n - 1) + n$
$T(n) = T(n - 3) + (n - 2) + (n - 1) + n$
Continuing,
$T(n) = 1 + 2 + 3 + \ldots + n$

Step 3: Apply the summation formula
$T(n) = n(n + 1) / 2$
$T(n) = (n^2 + n) / 2$

Step 4: Determine the tight bound
The dominant term is n^2.

Final Answer
$T(n) = \Theta(n^2)$

Justification: The sum of the first n positive integers is $n(n + 1)/2$, which grows quadratically.

Save Answer Next →

Q8 Substitution: Halving with Constant Work 4 marks

Prove that $T(n) = T(n/2) + 1$ is $\Theta(\log n)$, assuming n is a power of two.

Your Answer

Step 1: Given recurrence
$T(n) = T(n/2) + 1$

Step 2: Expand the recurrence
$T(n) = T(n/4) + 2$ $T(n) = T(n/8) + 3$
After k steps:
$T(n) = T(n/2^k) + k$

Step 3: Find the stopping point
$n/2^k = 1$
$n = 2^k$
$k = \log_2 n$

Step 4: Substitute
$T(n) = T(1) + \log_2 n$
Since $T(1)$ is a constant:
$T(n) = 1 + \log_2 n$

Final Answer
$T(n) = O(\log n)$
Justification: Each recursive call halves n, so the recursion depth is $\log_2 n$.

Save Answer Next →

Q9 **Substitution: Exponential Recurrence** 4 marks

Establish a tight asymptotic bound for T(n) = 2T(n−1) + 1 with T(0)=1. Explain the growth pattern.

Your Answer

Step 1: Given recurrence
T(n) = 2T(n − 1) + 1, where T(0) = 1.

Step 2: Expand the recurrence
T(n) = 2[2T(n − 2) + 1] + 1 T(n) = 4T(n − 2) + 3 T(n) = 8T(n − 3) + 7
After k steps:
T(n) = 2^k T(n − k) + (2^k − 1)

Step 3: Apply the base case
Set k = n:
T(n) = 2^n T(0) + (2^n − 1)
Since T(0) = 1:
T(n) = 2^n + 2^n − 1 T(n) = 2^(n+1) − 1

Step 4: Tight asymptotic bound
T(n) = Θ(2^n)

Justification: Each recursive call doubles the previous value, producing exponential growth.

Save Answer Next →

Q10 **Check a Complexity Claim** 4 marks

An analyst claims T(n) = T(n/2) + n is Θ(n log n). Assess the claim and establish the correct asymptotic bound.

Your Answer

Step 1: Given recurrence
T(n) = T(n/2) + n

Step 2: Identify parameters
a = 1, b = 2, f(n) = n

Step 3: Calculate
$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$
Since f(n) = n grows faster than 1, Master Theorem Case 3 applies.

Step 4: Establish the correct bound
The non-recursive work dominates, so:
T(n) = Θ(n)

Final Answer

The analyst's claim O(n log n) is correct but not tight. The tight asymptotic bound is T(n) = Θ(n).

Save Answer Next →

Q11 Algorithm Crossover 4 marks

Algorithm A performs $5n \log_2 n$ operations and Algorithm B performs $0.02n^2$ operations. Compare them and estimate the crossover input size.

Your Answer

Step 1: Given operations
Algorithm A: $T_1(n) = 5n \log_2 n$
Algorithm B: $T_2(n) = 0.02n^2$

Step 2: Find the crossover point
Set both operations equal:
$5n \log_2 n = 0.02n^2$
Divide by n:
$5 \log_2 n = 0.02n$
$n = 250 \log_2 n$

Step 3: Estimate
Solving numerically gives approximately:
$n \approx 3,600$

Step 4: Compare
For $n < 3,600$: Algorithm B generally performs fewer operations.
For $n > 3,600$: Algorithm A performs fewer operations.
Final Answer
The estimated crossover input size is $n \approx 3,600$. Algorithm A is more efficient for sufficiently large inputs because n log n grows more slowly than n^2.

Save Answer Next →

Q12 Scalability: n log n 4 marks

A program processes 10,000 records in 2 seconds using an $O(n \log_2 n)$ algorithm. Estimate runtime for 100,000 records, assuming proportional scaling and negligible fixed overhead.

Your Answer

Step 1: Given
Initial records, $n_1 = 10,000$
Initial runtime, $T_1 = 2$ seconds
New records, $n_2 = 100,000$
Complexity = $O(n \log n)$

Step 2: Apply proportional scaling
$T_2 = T_1 \times (n_2 \log_2 n_2) / (n_1 \log_2 n_1)$
$T_2 = 2 \times (100,000 \times \log_2 100,000) / (10,000 \times \log_2 10,000)$

Step 3: Calculate
$T_2 \approx 2 \times 10 \times (16.61 / 13.29)$
$T_2 \approx 20 \times 1.25$

Final Answer
Estimated runtime ≈ 25 seconds.
Justification: The number of records increases tenfold, but the logarithmic factor also increases, so runtime grows by approximately 12.5 times.

Save Answer Next →

Q13 **Scalability: Quadratic** 4 marks

An $O(n^2)$ algorithm takes 4 seconds for 2,000 items. Estimate runtime for 6,000 items, assuming the quadratic term dominates.

Your Answer

Step 1: Given
Initial input: $n_1 = 2{,}000$
Initial runtime: $T_1 = 4$ seconds
New input: $n_2 = 6{,}000$
Complexity: $O(n^2)$

Step 2: Apply the quadratic scaling formula
$T_2 = T_1 \times (n_2 / n_1)^2$
$T_2 = 4 \times (6{,}000 / 2{,}000)^2$

Step 3: Calculate
$T_2 = 4 \times 3^2$
$T_2 = 4 \times 9 = 36$ seconds

Final Answer
Estimated runtime = 36 seconds.
Justification: The input size triples, so the runtime increases by $3^2 = 9$ times for a quadratic algorithm.

Save Answer Next →

Q14 **Compare Three Growth Rates** 4 marks

Compare $O(n)$, $O(n \log n)$, and $O(n^2)$ growth. Discuss how their operation counts scale and give an appropriate use case for each.

Your Answer

Comparison of Time Complexity

Complexity	Operation Count	Growth	Use Case
$O(n)$	Proportional to n	Grows linearly	Finding an element using linear search
$O(n \log n)$	Proportional to $n \log n$	Grows moderately	Merge sort
$O(n^2)$	Proportional to n^2	Grows rapidly	Bubble sort

Explanation:
$O(n)$: If input doubles, operations approximately double.
$O(n \log n)$: If input doubles, operations increase slightly more than double.
$O(n^2)$: If input doubles, operations become approximately four times greater.

Conclusion: $O(n)$ is generally the most efficient, followed by $O(n \log n)$, while $O(n^2)$ becomes inefficient for large inputs.

Save Answer Next →

Q15 **Constant Factors and Big-O** 4 marks

Two algorithms take $T_1(n)=100n$ and $T_2(n)=n \log_2 n$. Explain why asymptotic notation alone may not determine which is faster for small inputs.

Your Answer

Asymptotic notation describes the growth rate for large inputs but ignores constant factors.
$T_1(n) = 100n$
$T_2(n) = n \log_2 n$

For small inputs, $T_2(n)$ may be faster because it has a smaller operation count. For example, when $n = 10$, $T_1 = 1000$ and $T_2 \approx 33$. However, actual speed also depends on implementation and hardware.

Conclusion: Asymptotic notation alone cannot determine which algorithm is faster for small inputs because constant factors and implementation overhead also matter.

Save Answer Next →

Q16 **Hash Set: Time and Space** 4 marks

An algorithm scans an array once and stores distinct values in a hash set. Analyze expected time and auxiliary space, stating assumptions about hash operations.

Your Answer

Expected Time Complexity: O(n), assuming hash set insertion and lookup take O(1) expected time per operation.

Auxiliary Space Complexity: O(n), because the hash set may store up to n distinct values.

Assumption: Hash operations take constant expected time, with a good hash function and few collisions. In the worst case, time can be $O(n^2)$ due to collisions.

Save Answer Next →

Q17 **Recursive Temporary Arrays** 4 marks

A recursive algorithm splits an array into two halves, processes both halves, and allocates a temporary array of size n at each call. Analyze time and peak auxiliary space, clarifying whether allocations overlap.

Your Answer

Time Complexity: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Peak Auxiliary Space: O(n) if temporary arrays are freed before recursive calls continue, or if only one temporary array per active level is allocated. If each recursive call retains its size-n array while both halves are processed, allocations overlap, and peak space can reach $O(n \log n)$.

Conclusion: Time is $O(n \log n)$; peak space depends on the allocation lifetime and whether temporary arrays overlap.

Save Answer Next →

Q18 **Nested Loops with a Condition** 4 marks

An algorithm has three loops from 1 to n, but the innermost statement executes only when $i+j \leq n$. Determine its time complexity and justify the count.

Your Answer

Assuming the condition is $i + j = n$, the time complexity is $O(n^2)$.

Justification:
Three loops run from 1 to n.
The innermost statement executes only when $i + j = n$.
For each i, there is at most one valid j.

Therefore, the statement executes O(n) times for each k value.
Since k runs n times, the total count is $n \times n = n^2$.

Final Answer: $O(n^2)$.

Save Answer Next →

Q19 **Matrix Allocation vs Traversal** 4 marks

A program allocates an $n \times n$ matrix but traverses only the main diagonal. Distinguish total space complexity from traversal time complexity.

Your Answer

Total Space Complexity: $O(n^2)$, because the program allocates an $n \times n$ matrix containing n^2 elements.

Traversal Time Complexity: O(n), because only the n elements on the main diagonal are visited.

Conclusion: Space complexity is $O(n^2)$, while traversal time complexity is O(n).

Save Answer Next →

Q20 **Binary Search: Time and Space** 4 marks

Compare iterative and recursive binary search in time and auxiliary space. Explain the role of recursion-stack overhead.

Your Answer

Iterative Binary Search:

Time Complexity: O(log n)

Auxiliary Space: O(1), because it uses loop variables without recursion.

Recursive Binary Search:

Time Complexity: O(log n)

Auxiliary Space: O(log n), because each recursive call adds a frame to the recursion stack.
Role of Recursion-Stack Overhead: Each recursive call stores parameters and local variables in the stack, requiring extra memory.
Therefore, iterative binary search uses less auxiliary space than recursive binary search.

Save Answer Next →

Q21 **Unequal Subproblem Sizes** 4 marks

Analyze $T(n)=T(n/3)+T(2n/3)+n$ using a recursion-tree argument. Explain the work per level and total tree depth.

Your Answer

Given: $T(n) = T(n/3) + T(2n/3) + n$

Using the Recursion Tree Method:

1. Work per level: The subproblem sizes add up to n, so the total work at each level is $O(n)$.
2. Tree depth: The longest path follows the subproblem of size $2n/3$. Therefore, the depth is $O(\log n)$.
3. Total work: $O(n)$ work per level × $O(\log n)$ levels.

Final Answer: $T(n) = O(n \log n)$.

Save Answer Next →

Q22 **Unequal Split with Linear Combine** 4 marks

A divide-and-conquer algorithm splits into subproblems of sizes $n/4$ and $3n/4$ and performs $\Theta(n)$ combining work. Determine its asymptotic time complexity using a suitable method.

Your Answer

Recurrence relation:

$T(n) = T(n/4) + T(3n/4) + O(n)$

Using the Recursion Tree Method:

Each level performs a total of $O(n)$ work.

The longest path follows the subproblem of size $3n/4$.

The recursion depth is $O(\log n)$.
Therefore,

Time Complexity = $O(n \log n)$.

Save Answer Next →

Q23 **Fibonacci-Style Recurrence** 4 marks

A recursive procedure satisfies $T(n)=T(n-1)+T(n-2)+\Theta(1)$. Determine its asymptotic growth and explain the source of the growth.

Your Answer

Given: $T(n) = T(n-1) + T(n-2) + O(1)$

Asymptotic Growth:

This recurrence follows the Fibonacci pattern.

Each call generates two recursive calls, causing the number of calls to grow exponentially.

The characteristic equation is $r^2 = r + 1$, whose largest root is $\varphi = (1 + \sqrt{5})/2 \approx 1.618$.

Final Answer: $T(n) = \Theta(\varphi^n) \approx \Theta(1.618^n)$.

Source of Growth: Repeated branching into two smaller subproblems causes exponential growth in the number of recursive calls.

Save Answer Next →

Q24 **Four Half-Size Subproblems** 4 marks

Design and analyze a recurrence for an algorithm that splits into four subproblems of size n/2 and spends Θ(n) combining their results.

Your Answer

Recurrence Relation:
$T(n) = 4T(n/2) + O(n)$

Using the Master Theorem:
$a = 4, b = 2, f(n) = n$
$n^{(\log_2 4)} = n^2$
$f(n) = O(n)$, which is smaller than n^2.

Therefore, by Master Theorem Case 1:

Final Answer: $T(n) = \Theta(n^2)$.

Save Answer Next →

Q25 **Time-Space Trade-off** 4 marks

One algorithm uses O(n) time and $O(n^2)$ space; another uses O(n log n) time and O(n) space. Discuss which is more appropriate in a memory-constrained environment.

Your Answer

The second algorithm is more appropriate for a memory-constrained environment.

Algorithm 1: O(n) time and $O(n^2)$ space. It is faster theoretically but requires more memory.

Algorithm 2: O(n log n) time and O(n) space. It uses less memory and has reasonable performance.
s
Conclusion: Choose Algorithm 2 because its linear space requirement makes it more suitable for systems with limited memory, even though it takes slightly more time.

Save Answer